

KV reuse only pays off when saved prefill survives placement, transfer, and control.

A transport-focused technical report on when reusable KV state improves LLM serving: how the serving stack finds compatible state, routes or materializes it, keeps it off the decode critical path, and decides when recomputation is still the safer path.

Scope: transport, tiers, direct paths, CXL, NIXL, Mooncake TE, Beluga, PrfaaS

Method: source-grounded synthesis plus clearly labeled qualitative illustrations

Audience: LLM serving, cloud infrastructure, networking, and memory-system readers

Prepared for source-grounded technical reading; pricing and product pages should be rechecked before reuse in external materials.

0. Abstract

Hit rate is an intermediate variable. Serving operators ultimately care about SLO-qualified goodput, P50/P95 TTFT, TPOT stability, GPU idle time, cost per useful token, and safe cross-request reuse.

KV reuse converts prefill compute into a transport and control problem. A reusable prefix or block is valuable only after the serving stack answers four questions: is the state compatible, where is it, how fast can it become engine-ready, and what is the fallback if the answer changes under load?

This report focuses on the **transport-facing layer** of KV reuse: external KV stores, multi-tier materialization, direct memory paths, CXL-like pooling, disaggregated prefill/decode handoff, and routing/control metadata that determine whether saved prefill remains visible as lower TTFT or higher goodput.

```
reuse_value ~= p_hit * prefill_compute_saved
- storage_cost
- p_fetch * transfer_cost
- lookup_and_control_cost
- repair_or_fallback_cost
- tail_latency_risk
- quality_and_governance_risk
```

REPORTED

3.8×

Throughput improvement in vLLM × Mooncake Store on realistic Codex/SWE-bench Pro agentic traces.

REPORTED

46×

P50 TTFT reduction in the same experiment.

REPORTED

1.7%

→ **92.2%**
Cache-hit-rate jump that makes the transport path first-order rather than incidental.

Evidence disclaimer. Quantitative values in this report are either cited from the named source, derived from explicitly stated assumptions, or marked as illustrative. Illustrative curves and movement lower bounds are used to make the system trade-off tangible; they are not presented as measured end-to-end benchmark results.

1. Data scope and methodology

This is a transport-focused technical report, not a comprehensive academic survey. It selects systems and papers that change how KV state is placed, moved, materialized, routed, or governed across service and memory boundaries.

Scope boundary

In scope	External KV stores, prefill/decode handoff, distributed KV transfer, CXL or remote-memory pooling, direct GPU paths, KV-aware routing, metadata/ABI requirements, and transport-visible governance.
Context only	Local engine layout, prompt/prefix caching, compression/retention, agent workflow reuse, and cross-model reuse are included only when they change transport demand or compatibility requirements.
Out of scope	A complete taxonomy of all KV compression, eviction, semantic caching, transformer-efficiency, MoE, compiler, or agent-framework work.

Evidence handling

Reported	Numbers copied or paraphrased from the cited paper, product document, or project blog, with the original workload and assumptions kept visible where possible.
Derived	Simple calculations from stated quantities, such as KV object size divided by peak bandwidth. These are lower bounds unless otherwise stated.
Sourced inference	Architecture interpretation supported by source text but not usually stated as a single sentence by the source.
Illustrative	Qualitative curves or diagrams used to explain trade-offs. They guide intuition rather than replacing measurement.

Reading rule. Compare systems by the boundary they move and the metric they optimize. A direct handoff mechanism, a CXL memory pool, and a cross-cluster prefill service are not substitutes; they occupy different points in the transport/control design space.

2. Claim and evidence ledger

The ledger makes the report auditable: each major claim is tied to an evidence type, a source trail, and the condition under which the claim should be read.

Claim used in this report	Evidence type	Condition / caveat	Source
KV reuse is best judged by SLO-qualified goodput, TTFT/TPOT stability, and fleet utilization rather than hit rate alone.	Sourced inference	Derived from serving-oriented papers and product systems where reuse must survive queueing, transfer, and scheduling.	S1, S3, S7, S9
Distributed KV pooling can sharply increase hit rate for agentic traces and expose transport as a first-order bottleneck.	Reported	vLLM × Mooncake Store reports 1.7% to 92.2% hit-rate movement, 3.8× throughput, and 46× P50 TTFT improvement under its stated workload.	S1
A 100K-token Kimi-style FP8 KV object is large enough that movement cost can dominate a request path.	Reported + derived	The 3.8 GB anchor is model- and representation-specific; capacity-equivalent calculations inherit that assumption.	S1, S16
Public cached-input discounts are strong economic signals for reuse but not proof of a specific internal KV implementation.	Reported pricing + interpretation	OpenAI and DeepSeek pricing pages demonstrate commercial value assigned to reused input state; they do not expose provider internals. Exact prices are point-in-time and should be rechecked.	S4, S5, S6
Mooncake TE is best read as a transfer-plan substrate plus metadata/store control, not just a raw RDMA wrapper.	Sourced inference	The transport gain depends on segment abstraction, batch transfer, endpoint choice, and metadata orchestration.	S2, S3
CXL pooling is not automatically KV-aware; Beluga adds KV indexing, layout conversion, and access orchestration above the memory substrate.	Sourced inference	The distinction matters because raw load/store memory does not by itself know model, block, RoPE, or tier semantics.	S8
Dynamo/NIXL is strongest as a direct prefill-to-decode transfer path within an NVIDIA-aligned serving stack.	Reported + interpretation	The report treats it as focused direct handoff rather than a full long-lived multi-tier KV lifecycle.	S7
PrfaaS is viable only when remote prefill savings exceed queueing, returned-KV transfer, metadata, and tail-risk costs.	Sourced inference	The boundary shift depends on selective admission, bandwidth awareness, and model-side KV footprint reduction.	S9

3. Why this became a systems topic

KV-cache work has moved from local memory layout into a transport-facing systems problem: reusable state now crosses service, tier, host, rack, and sometimes model/workflow boundaries. The figure is a scoped signal map, not a bibliometric claim.

Scoped signal map: KV cache becomes multi-branch serving infrastructure

Indicative lower-bound counts in this report corpus. The point is branching across memory, transport, routing, and workflow boundaries.



Reading rule: the bar heights show scoped corpus density; the branch labels show why transport/control became first-order.

Figure 1. A full-width signal map makes the branch-out visible: after 2024, KV work is no longer only allocation and prefix caching; it reaches stores, direct paths, routing policy, CXL-like memory, and agentic workflows.

Local substrate

Paged KV, radix caches, prefix cache. The object is still engine-local.

State service

External KV pool, distributed metadata, and hot/warm/cold tiers make reuse a service.

Transport substrate

RDMA, GPUDirect, NIXL, NVMe-oF, and CXL-like semantics decide materialization cost.

Control plane

Cache-aware routing, load balancing, SLO admission, and future-use hints decide the action.

Workflow boundary

Agent turns, tool stalls, shared workspaces, and cross-model compatibility widen reuse.

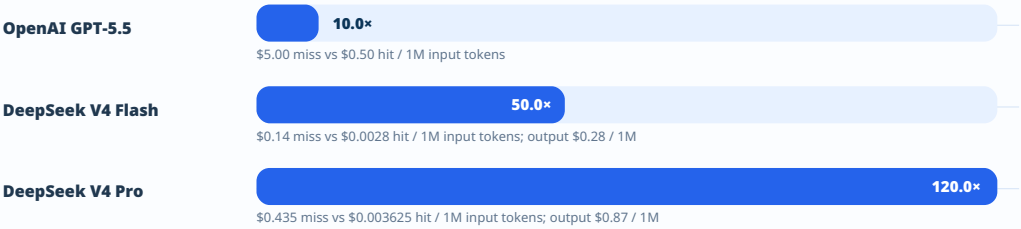
Earlier KV work mainly optimized live local memory: allocation, paging, retention, and repeated prompt fragments. Recent work makes KV cross service boundaries: prefill to decode, engine to external store, GPU to DRAM or SSD, pod to pod, and in some proposals model or agent to model or agent.

4. Economic motivation: cache-friendly behavior is already priced.

Public cached-input discounts validate the economic surface: providers value reuse. They do not by themselves prove any specific internal implementation such as raw KV exposure, cross-node KV transfer, or a particular memory tier.

Input cache-miss price / input cache-hit price

Point-in-time official pricing. Economic signal only; not an implementation claim.



DeepSeek prices are from the official Models & Pricing page; prices are per 1M tokens and may change. Use this chart as an operator-economics motivation, not as evidence of an exposed KV API or a specific backend design.

Figure 2. The full-width pricing plot makes the cache-hit economic signal immediately visible. It motivates why providers and customers care about reuse, while remaining deliberately agnostic about internal implementation.

The economics of cached input creates a direct design pressure on application frameworks: stable system prompts, tool schemas, shared documents, and workspace memory should be made cacheable whenever policy permits. But provider pricing is only a market-facing surface. A report about serving architecture should avoid reading it as evidence of a specific internal KV transport mechanism.

Use pricing to motivate why customers and providers care. Use system papers and product docs to argue how reuse may be implemented inside a serving stack. DeepSeek prices below are taken from the official Models & Pricing page, where prices are stated per 1M tokens and may vary over time.

\$4

\$5

Provider / model	Input hit	Input miss	Miss / hit
OpenAI GPT-5.5	\$0.50 / 1M	\$5.00 / 1M	10.0×
DeepSeek V4 Flash	\$0.0028 / 1M	\$0.14 / 1M	50.0×
DeepSeek V4 Pro	\$0.003625 / 1M	\$0.435 / 1M	120.0×

Output prices are \$30.00 / 1M for OpenAI GPT-5.5 in the cited OpenAI pricing page, and \$0.28 / 1M / \$0.87 / 1M for DeepSeek V4 Flash / Pro in the cited DeepSeek pricing page.

5. Operator decision model: route, move, repair, compress, or recompute.

The central policy question in KV reuse is the action policy. A controller chooses the cheapest safe state action under SLO, not maximize nominal hit rate.

Situation	Likely better action	Transport/control requirement	Common failure mode
Hot compatible KV is already resident near a lightly loaded worker.	Route request to state.	Fresh location index, load signal, tenant policy, cache-affinity scoring.	Affinity creates hotspots and hurts SLO.
KV is large, compatible, and near enough to arrive before recompute.	Move state to request.	Memory descriptor, batch transfer plan, background I/O, completion event.	Transfer tail exceeds saved prefill.
KV is cold, fragmented, or remote, but future reuse is probable.	Prefetch or demote asynchronously.	Workflow hint, TTL, tier score, queue-aware admission.	Pollutes warm tier or moves unused bytes.
KV match is approximate or only partly compatible.	Repair selectively.	Deviation score, recompute boundary, quality fallback, provenance.	Quality drift or hidden correctness loss.
Decode loop is live and token-by-token latency dominates.	Keep live state local; avoid remote per-token fetch.	Hot HBM allocation, retention/compression policy, decode admission.	Remote read enters the critical loop and destroys TPOT.
Tenant, model, tokenizer, RoPE, dtype, or layout mismatches.	Reject reuse and recompute.	Compatibility ABI, validity epoch, policy gate, auditable fallback.	Unsafe reuse or data leakage.

6. Structural lens: compile reusable context into SLO-aware state actions.

The cross-layer story is not a loose ecosystem map. It is a compilation path: application and model structure produce candidate KV objects; the controller validates and chooses actions; memory and transport substrates make those actions feasible or too expensive.

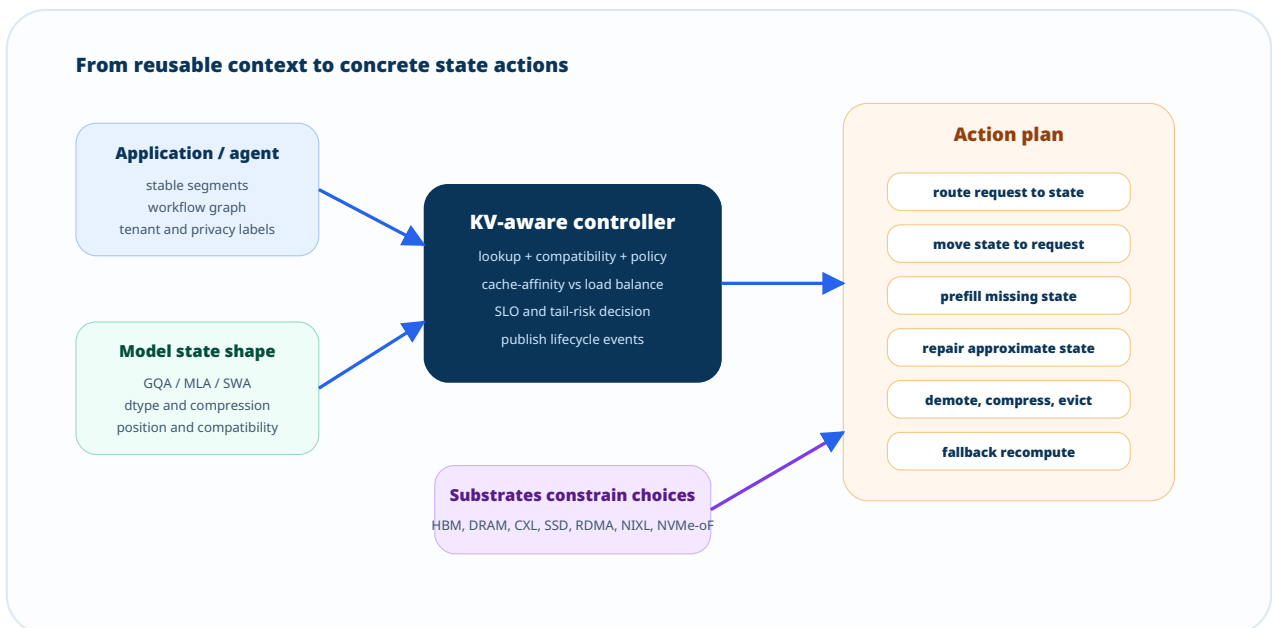


Figure 3. Structural lens: application-visible reusable context and model state shape are compiled into concrete state actions by a KV-aware controller; substrates constrain which action is actually cheap.

7. Request journey: every hit creates a materialization obligation.

Transport is not a postscript after scheduling. It constrains which cache-aware decision is good because a late, fragmented, CPU-bound, or unsafe hit can reduce goodput while increasing nominal hit rate.



Figure 4. Request journey: a cache hit becomes useful only after lookup, compatibility checking, action selection, materialization, decode binding, and lifecycle publication.

Decode nuance. KV reuse mainly saves prefill. Decode benefits indirectly when reuse keeps prefill from competing with decode, places the request near needed state, reduces live working-set pressure, and avoids transferring large context into the token-by-token path. For long-output workloads, decode is a retention and bandwidth problem more than a classic prefix-hit problem.

SLO tax. In this report, SLO tax means the extra lookup, queueing, descriptor setup, transfer, validation, retry, and tail-latency risk introduced by materializing reusable state.

8. Quantitative case: larger pools raise hit rate; deeper tiers add movement tax.

A transport-centered reading needs two quantitative axes together: reachable KV capacity can improve hit rate for agentic traces, while movement across more tiers adds latency, control work, and tail risk.

REPORTED

3.8 GB

Approximate Kimi-2.5 FP8 KV footprint for a 100K-token context in the cited vLLM × Mooncake analysis.

S1

DERIVED LOWER BOUND

76 ms

Ideal transfer time for a 3.8 GB object over a 400 Gb/s link before protocol, queueing, registration, and fragmentation overheads.

ILLUSTRATIVE

hit gain – movement tax

The central trade-off: larger reachable pools raise hit opportunity, but deeper tiers can push materialization into the TTFT tail.

Quantitative disclaimer. Capacity-equivalent calculations use the Kimi-style 3.8 GB / 100K-token anchor only to create intuition. They are not a generic model-size law; different models, GQA/MLA choices, precision, compression, and block granularity change the object size.

8.1 Grounded sizing anchor: Kimi-style agentic serving

The vLLM × Mooncake analysis uses Kimi-2.5 on GB200 nodes and states that a 100K-token context can occupy about 3.8 GB for Kimi-2.5 FP8 KV caches. This is a useful anchor, but it is model-specific; it should not be generalized to all LLMs.

S1

S16

```
Kimi_style_KV_GB(context) ~= 3.8 GB * context_tokens / 100000

20K tokens -> 0.76 GB KV
80K tokens -> 3.04 GB KV # representative late-turn trace size
180K tokens -> 6.84 GB KV # long-context tail
600K tokens -> 22.8 GB KV # stress point, not a trace average
```

Tier / resource view	Capacity anchor	100K-prefix equivalents	Serving interpretation
Single GPU HBM	H200: 141 GB HBM3e; B300-class product pages advertise up to 288 GB HBM per GPU.	After weights and runtime reserve, only a fraction is available for persistent shared KV.	Best for live hot state; weak as an agentic shared pool. S13 S14
Server DRAM	Beluga discusses 2 TB local DRAM per server as a representative RDMA-pool contribution.	2 TB / 3.8 GB ~= 526 100K-prefix equivalents.	Large warm tier, but copy path, PCIe, RDMA setup, and fragmentation can dominate TTFT. S8
Rack-scale memory	GB300 NVL72 product pages list up to 21 TB HBM3e and 17 TB LPDDR5X per rack.	17 TB / 3.8 GB ~= 4473 100K-prefix equivalents before replication and fragmentation.	Large enough to make prefix reuse a fleet control-plane problem. S14
CXL / remote DRAM / SSD	Capacity can grow further through CXL pools and distributed KV stores.	Capacity rises; latency distribution also widens.	The metric becomes hit-rate gain minus restore tail, not capacity alone.

8.2 Hit-rate side of the trade-off

Reachable KV pool size → cache hit → prefill goodput

Denominator: request-level prefix hit in reported vLLM + Mooncake experiment; intermediate curve is illustrative.

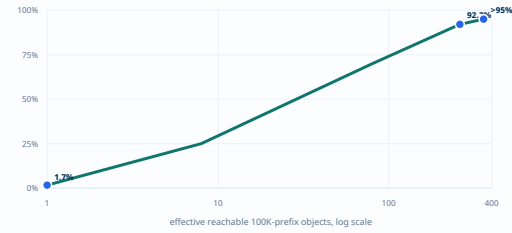


Figure 5. Hit-rate intuition: increasing the effective reachable KV pool can convert repeated agentic prefixes from mostly missed local state into reusable prefill savings.

Interpretation

On Codex-style agentic traces, a local/system-prompt-only cache can miss most of the reusable state. A distributed pool increases the reachable working set across turns and instances. This changes the denominator of prefill work: the system now often prefill-computes only the new delta instead of the entire accumulated context.

A simple prefill-only intuition: if 90% of a turn is reusable prefix, then hit rate $h=1.7\%$ leaves about 98.5% of prefill work, while $h=92.2\%$ leaves about 17.0%. That is roughly 5.8× theoretical prefill-work reduction before queueing, decode, and transfer overheads. The reported 3.8× throughput gain is therefore plausible and also shows that hit rate is not the final metric.

8.3 Movement side of the trade-off

Interpretation

The movement plot uses the same 3.8 GB 100K-token Kimi-style object. Bars are lower bounds from peak bandwidth division, not measured TTFT. Real systems add block lookup, transfer setup, registration, scatter-gather fragmentation, endpoint selection, retries, interference, and tail queueing.

This is why tier depth has to be coupled with policy. More storage can raise p_{hit} , but it also raises p_{fetch} and transfer_cost unless route-to-state, prefetch, batching, and direct paths keep the object out of the request's critical path.

Movement lower bound for one 100K-token Kimi-style KV object

Lower bound = 3.8 GB / peak bandwidth. Real p95 can be much higher.



Figure 6. Movement lower bounds: larger and deeper KV pools increase reuse opportunity, but the movement path can add enough latency and variance to erase the hit benefit.

9. KV object ABI: make state visible before making transport clever.

The transport layer cannot optimize anonymous tensor bytes. It needs a minimal KV object contract that exposes identity, compatibility, location, validity, policy, and completion semantics.

Field group	Representative fields	Why it matters
Identity	object id, block hash, request/session/workflow id, producer engine id.	Names reusable state and lets metadata services avoid duplicate discovery.
Model compatibility	model id, tokenizer id, architecture, layer/head layout, block size, RoPE/position offset.	Prevents invalid reuse across incompatible executions.
Representation	dtype, quantization, compression, sparsity, chunk/page format, endian/alignment.	Determines whether the destination engine can bind the state directly or must decode/repair.
Policy and governance	tenant, ACL, privacy label, TTL, provenance, confidence, quality-risk class.	Decides whether reuse is allowed, auditable, and reversible.
Location and memory descriptor	tier, address, length, registration state, NIC affinity, CXL region, NVMe object, fault domain.	Lets the transport construct a real movement plan instead of a blind RPC.
Lifecycle and telemetry	validity epoch, owner, refcount, hit count, last access, queue delay, p95 transfer time, fallback reason.	Feeds eviction, routing, admission, and postmortem debugging.

Object identity

KV needs names at block/chunk/session scope before it can be routed, fetched, or shared.

Memory descriptor

HBM, DRAM, CXL, and NVMe are not interchangeable. The controller needs addressability, registration, permissions, and fault domains.

Transfer plan

Layer/head/page fragments become batched scatter-gather plans with async completion and fallback semantics.

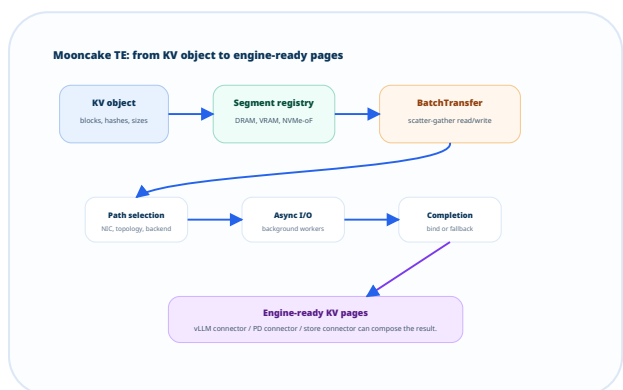
10. Mooncake TE as a transfer-plan substrate.

Mooncake TE is best read as more than RDMA. RDMA is one backend; the higher-level contribution is registered segments, asynchronous batch transfers, topology-aware path selection, and a substrate that Mooncake Store or vLLM connectors can compose into KV materialization.

What it adds above raw network transport

- **Segments:** named address spaces over RAM, VRAM, and NVMe-oF-like storage endpoints.
- **BatchTransfer:** asynchronous read/write batches over non-contiguous regions.
- **Backend diversity:** documented TCP, RDMA/ GPUDirect RDMA, EFA, NVMe-oF, NVLink, and HIP paths.
- **Path management:** NIC preference, multi-NIC pooling, slicing of larger transfers, endpoint pooling, retry and completion semantics.

The vLLM integration then uses the Mooncake master for KV-block metadata and client health, workers for background movement, and GPUDirect RDMA for SM-free, zero-copy KV transfer where applicable.



S1

S2

Figure 7. Mooncake TE separates KV object intent from concrete transfer execution: the useful abstraction is the transfer plan, not a single RDMA copy.

Layer	What TE contributes	What still sits above TE
Data path	Registered segments, batching, backend selection, async transfer.	Which KV should move, quality policy, and tenant-safe compatibility.
Store integration	Moves KV between GPU HBM and distributed DRAM/SSD-like resources.	Master metadata, block hashes, admission, eviction, health, route policy.
Operator value	Lowers $p_fetch * transfer_cost$ and reduces CPU/GPU interference.	Control plane must keep requests from fetching cold or low-value state.

Case-study lens

Problem	Make external KV state materializable across GPU, DRAM, SSD, and network paths without turning every cache hit into CPU-bound copy work.
Boundary moved	Engine-local KV becomes a distributed KV service object that can support PD handoff and cross-request reuse.
Data path	Registered segments, asynchronous batch transfer, endpoint selection, and GPUDirect RDMA where applicable.
Control dependency	KV-block metadata, hash lookup, worker health, cache admission, eviction, and cache-aware routing above the transfer engine.
Evidence	Production paper plus vLLM integration report; reported numbers are read under the stated Kimi/Codex/SWE-bench Pro conditions.
Best fit	Cluster-level KV pooling, repeated or agentic prefixes, PD serving, and multi-tier KV materialization.
Primary stress point	Tail latency under contention, metadata freshness, tenant policy, and whether transfer completion arrives before recomputation would have won.
Not a claim	TE by itself does not define the full compatibility, governance, or quality policy for every reuse scenario.

11. Beluga: CXL is the substrate; Beluga-KVCache is the KV-aware layer.

Beluga is clearest when framed in two steps: it argues that RDMA-style movement is awkward for fragmented KV, and it adds KV-aware indexing/copy/control on top of generic CXL memory semantics. CXL pooling alone is not KV-block aware.

Beluga: memory semantics help, but KV awareness is added above CXL



The KV awareness is software above CXL, not an inherent property of a CXL switch.

Figure 8. Beluga framing: CXL provides shared-memory semantics, while KV awareness is supplied by indexing, layout handling, copy kernels, and scheduling above the substrate.

What Beluga adds on RDMA

Beluga argues that RDMA is a network protocol rather than a memory bus. CPU-driven paths can copy through bounce buffers, GPU-driven paths can require polling kernels and synchronization, and fragmented KV layouts can stress NIC scatter-gather limits. The paper gives a concrete Qwen-32B GQA example with 128 non-contiguous chunks per token, and sparser cases can create even more fragments.

S8

What Beluga adds on CXL pooling

CXL switch-attached memory gives a shared memory semantic, but it does not know the model, tokenizer, RoPE position, block layout, or tenant policy of a KV object. Beluga-KVCache adds the KVCache index, data copy kernels, shared-memory RPC path, and scheduling simplification needed to make the CXL pool KV-aware.

Question	Beluga answer	Practical constraint
Why memory semantics?	Load/store-like access can remove part of the verb/queue/ completion overhead; the paper reports microbenchmark latency gains over an RDMA baseline.	CXL bandwidth, switch contention, coherence scope, adapter count, and GPU-to-CXL path maturity remain deployment constraints.
Why not pure CXL pooling?	KV still needs block metadata, layout conversion, compatibility checks, and lifecycle policy.	The KV awareness is tailored software, not a native feature of the memory fabric.
Why not replace all RDMA?	Beluga is strongest as nearby shared memory for low-latency KV access.	RDMA remains natural for wider cluster and cross-cluster movement.

Case-study lens

Problem	RDMA-style movement is awkward when KV is large, fragmented, and frequently accessed across tiers.
Boundary moved	Remote memory pool becomes a CXL memory-semantic pool with a KV-aware software layer.
Data path	CXL-attached memory semantics plus KV index, gather/scatter-oriented access, copy kernels, and RPC/control paths.
Control dependency	Placement, memory allocation, layout conversion, block indexing, and scheduler integration above raw CXL memory.
Evidence	Preprint-level system proposal and evaluation; strong as an architectural direction, still dependent on CXL ecosystem maturity.
Best fit	Nearby pod/rack memory expansion where low-latency shared memory semantics can beat explicit movement overhead.
Primary stress point	CXL switch contention, GPU-to-CXL access path, coherence scope, deployment availability, and multi-tenant isolation.
Not a claim	CXL pooling is not automatically KV-aware; Beluga adds KV semantics above the substrate.

12. Dynamo / NIXL: focused direct handoff rather than a full KV lifecycle.

Dynamo's transport strength is production disaggregated serving with NIXL-based direct VRAM-to-VRAM KV transfer. It is strongest when KV is handed from prefill to decode inside a fast NVIDIA-oriented fabric.

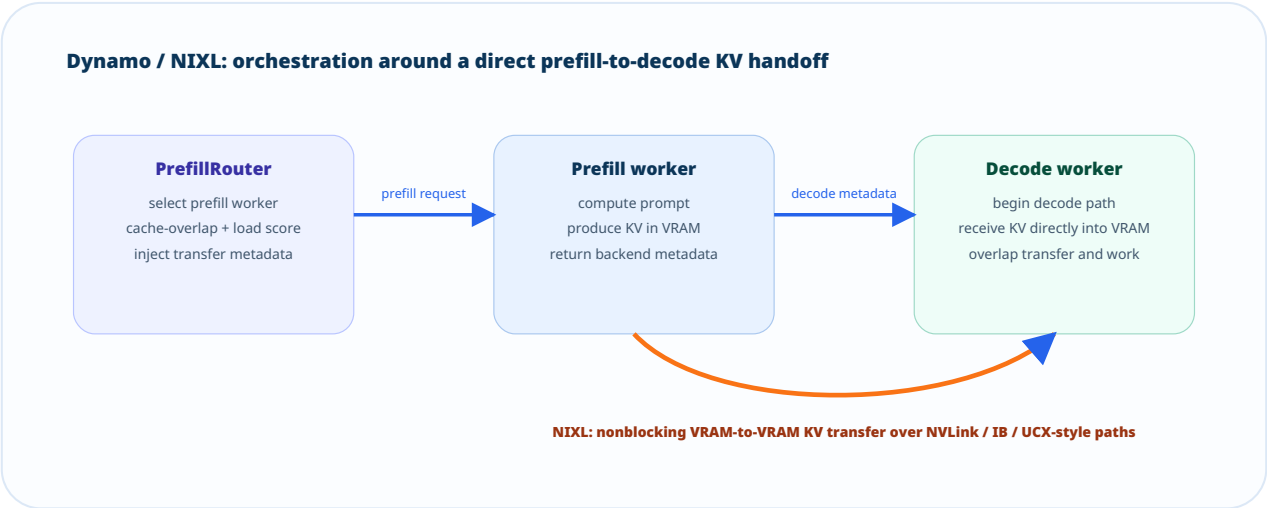


Figure 9. Dynamo/NIXL framing: the core value is direct, coordinated prefill-to-decode KV handoff inside a production disaggregated serving path. The Dynamo documentation describes disaggregated execution as prefill computing KV, KV transfer to decode, and decode token generation. It states that NIXL transfers KV directly from prefill VRAM to decode VRAM and that the transfer is nonblocking. The router uses cache overlap and load, injects backend-specific transfer metadata, and coordinates transfer over fabric backends.

Axis	Mooncake TE / Store	Dynamo / NIXL
Primary positioning	KV-centric distributed store plus multi-substrate transfer substrate.	Production disaggregated serving stack with direct KV handoff.
Best path	HBM/DRAM/NVMe-oF-like segments, RDMA/TCP/GDS-like movement, distributed pool.	Prefill VRAM to decode VRAM, nonblocking transfer inside fast GPU/network fabrics.
Strength	Broader external KV lifecycle: cache pool, offload, cross-instance prefix reuse.	Tight orchestration and native alignment with NVIDIA GPU/fabric ecosystem.
Potential gap	More policy and governance needed above transport.	Less obviously a general-purpose long-lived multi-tier KV store unless paired with other cache layers.

Case-study lens

Problem	In PD serving, prefill-produced KV must reach decode quickly without blocking the critical path or consuming avoidable GPU compute.
Boundary moved	Prefill/decode separation becomes a production direct-handoff path coordinated by the serving router.
Data path	NIXL-based direct prefill-VRAM to decode-VRAM transfer with backend-specific metadata and nonblocking transfer semantics.
Control dependency	Router cache-overlap scoring, load signals, transfer metadata injection, and decode worker coordination.
Evidence	Vendor documentation and product architecture; useful as production signal, not an independent academic benchmark by itself.
Best fit	NVIDIA-aligned disaggregated serving where the key boundary is fast prefill-to-decode handoff.
Primary stress point	Scope of external cache lifecycle, multi-tier persistence, and portability outside the assumed fabric/software stack.
Not a claim	NIXL direct handoff does not imply a complete long-lived distributed KV store unless paired with cache/storage layers.

13. PrfaaS: cross-cluster prefill is a conditional boundary shift.

PrfaaS matters because it asks how far the PD boundary can move. The answer is not always outward: remote prefill wins only when model-side KV shrinkage, selective admission, bandwidth-aware routing, and tail-risk control line up.

PrfaaS: cross-cluster prefill after returned state becomes small enough

Remote prefill is an admission-controlled action, not a default replacement for local prefill.

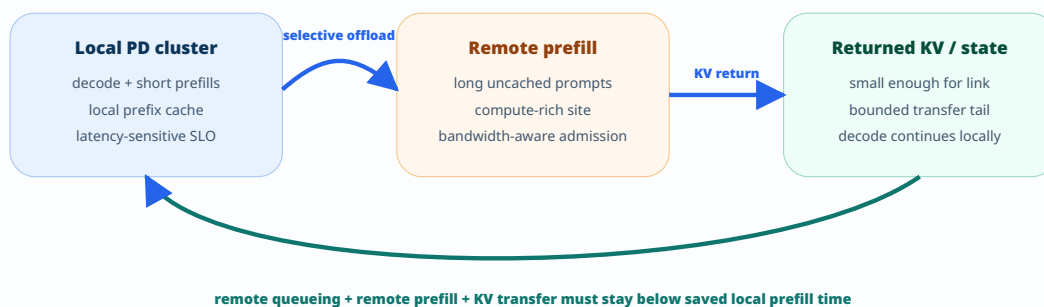


Figure 10. A larger PrfaaS diagram emphasizes the conditional nature of cross-cluster prefill: the returned state must be small enough and the end-to-end remote path must beat local prefill under tail-risk constraints.

PrfaaS starts from a clear condition: conventional dense-attention PD is tied to a high-bandwidth domain because prefill emits too much KV. Hybrid or KV-efficient attention can shrink the returned state and make commodity-Ethernet or cross-cluster service plausible for selected long-context requests.

S9

Condition	Why it matters	PrfaaS mechanism
Long uncached prefill	Short requests do not save enough compute to pay cross-cluster transfer.	Length-based threshold routing.
Small returned KV	Dense models can emit too much KV for commodity Ethernet.	KV-efficient hybrid attention and model-side reduction.
Bandwidth-aware admission	Cross-cluster links fluctuate and can become the bottleneck.	Bandwidth-aware scheduling before congestion accumulates.
Cache-aware placement	Uneven prefix caches can erase the benefit of remote prefill.	Global KV manager and hybrid prefix cache.

```
remote_prefill_is_worth_it when:

saved_local_prefill_time
> remote_queueing
+ remote_prefill_time
+ KV_transfer_time
+ metadata_control_overhead
+ tail_risk
+ fallback_risk
```

Read the formula as an operator decision rule. PrfaaS is not claiming that remote prefill is universally better; it is a selective action that becomes credible when returned state is compact and the network path is controlled.

Case-study lens

Problem	Prefill demand is bursty and heterogeneous; local clusters may have decode pressure while remote capacity could absorb long prefills.
Boundary moved	PD boundary expands from intra-cluster to cross-cluster or cross-site prefill service for selected requests.
Data path	Remote prefill plus returned KV/state over less specialized links, conditional on model-side KV shrinkage and bandwidth feasibility.
Control dependency	Length threshold, bandwidth-aware admission, global KV manager, cache placement, and tail-risk fallback.
Evidence	Preprint-level deployment-architecture proposal; most convincing under selective long-context admission rather than universal offload.
Best fit	Long uncached prompts, bursty prefill, heterogeneous clusters, and KV-efficient model architectures.
Primary stress point	WAN/cross-cluster queueing, bandwidth variability, returned-state size, and recovery when remote path misses SLO.
Not a claim	Remote prefill is not generally better; it is a conditional action selected only when the inequality closes.

14. Comparative map: compare by boundary moved, not by one speedup number.

Mooncake TE, Beluga, Dynamo/NIXL, and PrfaaS answer different transport questions. A useful report compares their boundary, data-path contribution, control-plane dependency, best-fit workload, failure mode, and evidence type.

Work	Boundary moved	Data-path proposition	Control-plane dependency	Best-fit regime	Evidence type	Stress point / non-goal
Moon cake TE / Store	Engine-local KV to distributed multi-tier KV service.	Segments plus BatchTransfer over heterogeneous memory/storage/network paths.	Metadata master, block hashes, health, admission, eviction, route policy.	Cluster KV pool, PD, agentic prefix reuse.	Paper + integration report.	Tail under contention; correctness and governance above the TE layer.
Beluga	RDMA memory pool to CXL memory-semantic KV pool.	Load/store-like access plus KV index, gather/scatter kernels, CXL RPC.	Block index, layout conversion, CXL memory allocation, scheduler integration.	Nearby rack/pod memory expansion with low-latency KV access.	Preprint proposal.	CXL ecosystem maturity, switch contention, GPU-to-CXL path, coherence scope.
Dynamo / NIXL	Prefill/decode handoff to production direct VRAM transfer.	Nonblocking prefill-VRAM to decode-VRAM transfer coordinated by router metadata.	PrefillRouter, cache-overlap/load scoring, transfer metadata injection.	NVIDIA GPU fabrics, production PD serving, fast handoff.	Product documentation.	Not a complete external cache lifecycle unless paired with KV store/tiering layers.
Prfaas	PD boundary to cross-cluster prefill service.	Selective long-context offload plus returned KV over less specialized links.	Bandwidth-aware admission, length threshold, global KV manager.	KV-efficient models with bursty long prefills.	Preprint architecture.	Tail risk, bandwidth variability, cache distribution, selective admission errors.

15. Evaluation discipline and governance considerations

Every transport claim answers an operator question: does this mechanism improve SLO-qualified goodput after movement, control, quality, and governance costs?

Evaluation dimensions

Boundary moved	Local page, external store, PD handoff, CXL pool, cross-cluster service, or agent workflow.
Workload regime	Agentic turn traces, RAG/document reuse, long-output decode, multi-tenant prefixes, or synthetic stress.
Hit denominator	Request-level hit, block-level hit, byte-weighted hit, or prefill-compute-weighted hit.
Path metric	Mean/p95 transfer time, queueing, CPU overhead, GPU interference, descriptor setup, retry rate.
End metric	P50/P95 TTFT, TPOT stability, SLO goodput, cost per token, GPU utilization, quality retention.

Governance checklist

Tenant isolation	Object metadata must encode tenant, ACL, policy, and revocation behavior.
Compatibility safety	Reject on model/tokenizer/RoPE/dtype/layout mismatch unless repair is explicitly validated.
Provenance	Store producer, source context, validity epoch, quality confidence, and fallback path.
Staleness	TTL, version, and context mutation invalidate or downgrade reuse confidence.
Auditability	Each reused KV object leaves a trace: lookup, decision, transfer, bind, repair, fallback.

Threat model example. A KV object produced for one tenant or model version can be technically reachable but semantically unsafe. A transport layer that can move the object faster must still be gated by compatibility metadata, tenant policy, provenance, and a fallback path to recomputation.

Source trail and extended readings

Sources are grouped by role. Product pages, pricing pages, and hardware specifications are point-in-time signals and should be rechecked before quoting exact prices or product limits. Pricing sources in this file were checked on 2026-06-05.

System papers and preprints

1. **S3** - [Mooncake FAST'25 paper](#)
2. **S8** - [Beluga: CXL-Based Memory Architecture for LLM KVCache](#)
3. **S9** - [Prefill-as-a-Service paper](#)
4. **S11** - [Agentix NSDI'26 paper](#)
5. **S12** - [DroidSpeak NSDI'26 paper](#)
6. **S10** - [FAST 2026 proceedings index](#)

Project and product documentation

1. **S1** - [vLLM × Mooncake Store blog, 2026](#)
2. **S2** - [Mooncake Transfer Engine design docs](#)
3. **S7** - [NVIDIA Dynamo disaggregated serving docs](#)
4. **S16** - [vLLM Kimi-K2.5 recipe](#)

Pricing and hardware signals

DeepSeek V4 Flash/Pro pricing was checked against the official Models & Pricing page for this report. Prices remain point-in-time signals.

1. **S4** - [OpenAI API pricing](#)
2. **S5** - [DeepSeek API Models & Pricing](#)
3. **S6** - [DeepSeek Context Caching guide](#)
4. **S13** - [NVIDIA H200 product page](#)
5. **S14** - [Supermicro GB300 NVL72 rack specifications](#)
6. **S15** - [NVIDIA ConnectX-7 datasheet](#)